Name: Asmit Pandey

Roll No: 2401072

Batch: T21 [AIDS]

Experiment 2: To Containerize Application using Docker

Aim:

To containerize application using Docker

This experiment maps to LO4.

LO4: Containerize, orchestrate and deploy applications using Docker, Docker Compose, Kubernetes and cloud infrastructure tools.

Theory:

Steps to install docker:

i. Navigate to the official Docker website (docker.com) and download the Docker Desktop installer for your operating system (Windows).
ii. Locate the downloaded installer file on your computer and double-click it to run the setup wizard.
iii. Follow the on-screen prompts. When asked, ensure the option to Use WSL 2 instead of Hyper-V is checked (for optimal performance on Windows).
iv. Wait for the installation to finish, click "Close," and restart your computer if the installer prompts you to do so.
v. After restarting, search for "Docker Desktop" in your system's Start menu and launch the application. Accept the service agreement to start the Docker engine.
vi. Open your terminal or PowerShell and type docker --version and press Enter. If it returns a version number, Docker is successfully installed and ready to use.

Docker:

Docker is an open-source platform that allows you to package an application and all its required dependencies into a single, standardized unit for software development. This ensures that the application runs exactly the same way regardless of where it is deployed—whether on your personal laptop, a testing environment, or a live production server. Instead of dealing with the old problem of "it works on my machine but not yours," Docker isolates the software so it behaves predictably everywhere.

Docker Images: A Docker image is a lightweight, standalone, and immutable (read-only) executable package. In theory, it is a static file that contains the complete set of instructions and components necessary to run a specific application.

An image is built from multiple stacked layers, where each layer represents a specific instruction—such as installing a dependency, setting an environment variable, or copying

application code. It encompasses the application code itself, the runtime environment, system tools, system libraries, and default configuration settings. Because it is read-only, an image cannot be changed once it is built.

Docker Containers: A container is the active, runtime execution of a Docker image. It is a standardized, isolated operating environment where the application actually runs and processes data.

In theory, when an image is executed, the Docker engine adds a thin, writable layer on top of the static image layers. This writable layer allows the active container to modify files, generate logs, and maintain a temporary state while it is running. Containers isolate the running software from the host operating system and from other containers, which ensures the application performs uniformly and predictably regardless of the underlying infrastructure.

Implementation:

1. docker --version

```
C:\Users\Lab 203>docker --version
Docker version 29.3.1, build c2be9cc
```

2. docker search Ubuntu

```
C:\Users\Lab 203>docker search ubuntu
NAME                              DESCRIPTION                                     STARS     OFFICIAL
ubuntu                            Ubuntu is a Debian-based Linux operating sys…   17859     [OK]
ubuntu/squid                      Squid is a caching proxy for the Web. Long-t…   129
ubuntu/nginx                      Nginx, a high-performance reverse proxy & we…   141
ubuntu/bind9                      BIND 9 is a very flexible, full-featured DNS…   120
ubuntu/kafka                      Apache Kafka, a distributed event streaming …   62
ubuntu/apache2                    Apache, a secure & extensible open-source HT…   104
ubuntu/zookeeper                  ZooKeeper maintains configuration informatio…   15
ubuntu/prometheus                 Prometheus is a systems and service monitori…   80
ubuntu/jre                        Chiseled Java runtime based on Ubuntu. Long-…   25
ubuntu/dotnet-aspnet              Chiselled Ubuntu runtime image for ASP.NET a…   26
ubuntu/python                     A chiselled Ubuntu rock with the Python runt…   33
ubuntu/mysql                      MySQL open source fast, stable, multi-thread…   74
ubuntu/postgres                   PostgreSQL is an open source object-relation…   45
ubuntu/redis                      Redis, an open source key-value store. Long-…   26
ubuntu/dotnet-runtime             Chiselled Ubuntu runtime image for .NET apps…   24
ubuntu/dotnet-deps                Chiselled Ubuntu for self-contained .NET & A…   16
ubuntu/grafana                    Grafana, a feature rich metrics dashboard & …   12
ubuntu/jdk                        Ubuntu rock for OpenJDK                         0
ubuntu/memcached                  Memcached, in-memory keyvalue store for smal…   7
ubuntu/rabbitmq                   Ubuntu-based RabbitMQ image - an open source…   0
ubuntu/mimir                      Ubuntu ROCK for Mimir, a horizontally scalab…   0
ubuntu/loki                       Grafana Loki, a log aggregation system like …   2
ubuntu/traefik                    Ubuntu ROCK for Traefik, a modern HTTP rever…   3
ubuntu/alertmanager               Ubuntu Rock for Alertmanager. It handles ale…   1
ubuntu/opentelemetry-collector    This is a rock for the OpenTelemetry Collect…   0
```

3. docker search ubuntu --filter stars=50

```
C:\Users\Lab 203>docker search ubuntu --filter stars=50
NAME                DESCRIPTION                                     STARS     OFFICIAL
ubuntu              Ubuntu is a Debian-based Linux operating sys…   17859     [OK]
ubuntu/squid        Squid is a caching proxy for the Web. Long-t…   129
ubuntu/nginx        Nginx, a high-performance reverse proxy & we…   141
ubuntu/bind9        BIND 9 is a very flexible, full-featured DNS…   120
ubuntu/kafka        Apache Kafka, a distributed event streaming …   62
ubuntu/apache2      Apache, a secure & extensible open-source HT…   104
ubuntu/prometheus   Prometheus is a systems and service monitori…   80
ubuntu/mysql        MySQL open source fast, stable, multi-thread…   74
```

4. docker search redis

```
C:\Users\Lab 203>docker search redis
NAME                              DESCRIPTION                                     STARS     OFFICIAL
mcp/redis                         Access to Redis database operations.            14
redis                             Redis is the world's fastest data platform f…   13604     [OK]
redis/redis-stack-server          redis-stack-server installs a Redis server w…   104
redis/redis-stack                 redis-stack installs a Redis server with add…   162
redis/redisinsight                Redis Insight - our best official GUI for Re…   54
redislabs/redis                   Clustered in-memory database engine compatib…   46
redis/rdi-api                                                                     0
redis/rdi-monitor                                                                 0
redis/rdi-operator                                                                0
redis/rdi-collector-initializer    Init container for RDI Collector               0
redis/rdi-collector-api                                                           0
redis/rdi-processor                                                               1
redis/rdi-cli                                                                     0
redis/rdi-metrics-aggregator                                                      0
redis/rdi-flink-processor                                                         0
redis/rdi-flink-collector                                                         0
redis/reloader                                                                    0
circleci/redis                    CircleCI images for Redis                       18
redis/redis-benchmark-go          redis-benchmark utility in go.                  1
redis/sidekiq-benchmark           Sidekiq load benchmark in Rust - throughput …   0
redis/vector-db-benchmark         Framework for benchmarking vector search eng…   1
bitnamicharts/redis               Bitnami Helm chart for Redis(R)                 37
redis/bcast-tracking-bench        Redis CLIENT TRACKING BCAST invalidation lis…   0
redis/redis-di-cli                                                                0
redis/pubsub-sub-bench                                                            0
```

5.

6. docker pull redis

```
C:\Users\Lab 203>docker pull redis
Using default tag: latest
latest: Pulling from library/redis
fd6b3803006b: Pull complete
c1af3958d6f8: Pull complete
8fd80e87c6f1: Pull complete
234dd6735104: Pull complete
6f0ec8fc933c: Pull complete
4f4fb700ef54: Pull complete
280ce3063e4e: Download complete
1bd39139fcfd: Download complete
Digest: sha256:234c902a2db49461a129e2d4aeff85b28cf20187ed274a67f6e50995fa713c7b
Status: Downloaded newer image for redis:latest
docker.io/library/redis:latest
```

7. docker run -d --name my_database redis

```
C:\Users\Lab 203>docker run -d --name my_database redis
d9334b4074e4ec3ca870fe3f16d05df5173f506a2e8e6f8375bc054e17401079
```

8. docker ps

```
C:\Users\15L>docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS      NAMES
9b0e3e996f29   redis     "docker-entrypoint.s…"   40 seconds ago   Up 40 seconds   6379/tcp   my_database
```

9. docker inspect my_database


```
C:\Users\Lab 203>docker inspect my_database
[
    {
        "Id": "d9334b4074e4ec3ca870fe3f16d05df5173f506a2e8e6f8375bc054e17401079",
        "Created": "2026-07-22T09:06:24.57150915Z",
        "Path": "docker-entrypoint.sh",
        "Args": [
            "redis-server"
        ],
        "State": {
            "Status": "running",
            "Running": true,
            "Paused": false,
            "Restarting": false,
            "OOMKilled": false,
            "Dead": false,
            "Pid": 3762,
            "ExitCode": 0,
            "Error": "",
            "StartedAt": "2026-07-22T09:06:24.75685398Z",
            "FinishedAt": "0001-01-01T00:00:00Z"
        },
        "Image": "sha256:234c902a2db49461a129e2d4aeff85b28cf20187ed274a67f6e50995fa713c7b",
        "ResolvConfPath": "/var/lib/docker/containers/d9334b4074e4ec3ca870fe3f16d05df5173f506a2e8e6f8375bc054e17401079/resolv.conf",
        "HostnamePath": "/var/lib/docker/containers/d9334b4074e4ec3ca870fe3f16d05df5173f506a2e8e6f8375bc054e17401079/hostname",
        "HostsPath": "/var/lib/docker/containers/d9334b4074e4ec3ca870fe3f16d05df5173f506a2e8e6f8375bc054e17401079/hosts",
        "LogPath": "/var/lib/docker/containers/d9334b4074e4ec3ca870fe3f16d05df5173f506a2e8e6f8375bc054e17401079/d9334b4074e4ec3ca870fe3f16d05df5173f506a2e8e
6f8375bc054e17401079-json.log",
        "Name": "/my_database",
        "RestartCount": 0,
        "Driver": "overlayfs",
        "Platform": "linux",
        "MountLabel": "",
        "ProcessLabel": "",
        "AppArmorProfile": "",
        "ExecIDs": null,
        "HostConfig": {
            "Binds": null,
            "ContainerIDFile": "",
            "LogConfig": {
                "Type": "json-file",
```


10. docker logs -f my_database

```
C:\Users\Lab 203>docker logs -f my_database
Starting Redis Server
1:C 22 Jul 2026 09:06:25.046 * oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
1:C 22 Jul 2026 09:06:25.046 * Redis version=8.8.0, bits=64, commit=00000000
, modified=1, pid=1, just started
1:C 22 Jul 2026 09:06:25.046 * Configuration loaded
1:M 22 Jul 2026 09:06:25.046 * monotonic clock: POSIX clock_gettime
1:M 22 Jul 2026 09:06:25.048 * Running mode=standalone, port=6379.
1:M 22 Jul 2026 09:06:25.048 * <bf> RedisBloom version 8.8.0 (Git=unknown)
1:M 22 Jul 2026 09:06:25.048 * <bf> Registering configuration options: [
1:M 22 Jul 2026 09:06:25.048 * <bf>      { bf-error-rate       :      0.01 }
1:M 22 Jul 2026 09:06:25.048 * <bf>      { bf-initial-size     :       100 }
1:M 22 Jul 2026 09:06:25.048 * <bf>      { bf-expansion-factor :         2 }
1:M 22 Jul 2026 09:06:25.048 * <bf>      { cf-bucket-size      :         2 }
1:M 22 Jul 2026 09:06:25.048 * <bf>      { cf-initial-size     :      1024 }
1:M 22 Jul 2026 09:06:25.048 * <bf>      { cf-max-iterations   :        20 }
1:M 22 Jul 2026 09:06:25.048 * <bf>      { cf-expansion-factor :         1 }
1:M 22 Jul 2026 09:06:25.048 * <bf>      { cf-max-expansions   :        32 }
1:M 22 Jul 2026 09:06:25.048 * <bf> ]
1:M 22 Jul 2026 09:06:25.048 * Module 'bf' loaded from /usr/local/lib/redis/
modules//redisbloom.so
1:M 22 Jul 2026 09:06:25.059 * <search> search-workers default: 12 (min of M
AX_WORKER_THREADS=16 and CPU cores)
1:M 22 Jul 2026 09:06:25.059 * <search> Redis version found by RedisSearch :
 8.8.0 - oss
1:M 22 Jul 2026 09:06:25.059 * <search> RediSearch version 8.8.0 (Git=d2ae02
6)
1:M 22 Jul 2026 09:06:25.059 * <search> Low level api version 1 initialized
successfully
1:M 22 Jul 2026 09:06:25.059 * <search> gc: ON, prefix min length: 2, min wo
rd length to stem: 4, prefix max expansions: 200, query timeout (ms): 500, t
imeout policy: return, oom policy: return, cursor read size: 1000, cursor ma
x idle (ms): 300000, max doctable size: 1000000, max number of search result
s:  1000000, default scorer: BM25STD,
1:M 22 Jul 2026 09:06:25.059 * <search> Initialized thread pools!
1:M 22 Jul 2026 09:06:25.059 * <search> Enabled workers threadpool of size 1
2
1:M 22 Jul 2026 09:06:25.066 * <search> Subscribe to config changes
1:M 22 Jul 2026 09:06:25.066 * <search> Subscribe to cluster slot migration
```

11. docker exec -it my_database sh

```
C:\Users\Lab 203>docker exec -it my_database sh
# exit
```

12. docker stop my_database

```
C:\Users\Lab 203>docker stop my_database
my_database
```

13. docker ps --all

```
C:\Users\Lab 203>docker stop my_database
my_database

C:\Users\Lab 203>docker ps -all
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS
                  PORTS     NAMES
d9334b4074e4   redis     "docker-entrypoint.s…"   4 minutes ago   Exited (0)
 23 seconds ago             my_database
```

14. docker rm my_database

    docker rmi redis

```
C:\Users\Lab 203>docker rm my_database
my_database

C:\Users\Lab 203>docker rmi redis
Untagged: redis:latest
Deleted: sha256:234c902a2db49461a129e2d4aeff85b28cf20187ed274a67f6e50995fa713c7b
```

15. docker images

```
C:\Users\Lab 203>docker images

                                                    i Info →   U  In Use
IMAGE                          ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-docker:latest            50ffa1e3a0bd       1.6GB          411MB    U
hello-world:latest             c3cbe1cc1aa5      25.9kB         9.49kB    U
new_img_name:latest            d0f28c79138d       202MB         53.2MB
nginx:latest                   5a88c9c45479       241MB           66MB
sample69:latest                9fc8bb8bc4fe      1.57GB          395MB    U
ubuntu:20.04                   8feb4d8ca535       111MB         29.3MB    U
ubuntu:latest                  3131b4cc82a7       161MB         45.3MB    U
```

16. docker search nginx

```
C:\Users\Lab 203>docker search nginx
NAME                                      DESCRIPTION                                     STARS     OFFICIAL
nginx                                     Official build of Nginx.                        21340     [OK]
nginx/nginx-ingress                       NGINX and  NGINX Plus Ingress Controllers fo…  122
nginx/nginx-prometheus-exporter           NGINX Prometheus Exporter for NGINX and NGIN…   52
nginx/nginx-ingress-operator              NGINX Ingress Operator for NGINX and NGINX P…   4
nginx/nginxaas-loadbalancer-kubernetes                                                    1
bitnamicharts/nginx                       Bitnami Helm chart for NGINX Open Source        4
ubuntu/nginx                              Nginx, a high-performance reverse proxy & we…   141
kasmweb/nginx                             An Nginx image based off nginx:alpine and in…   9
rancher/nginx                                                                             4
linuxserver/nginx                         An Nginx container, brought to you by LinuxS…   236
dtagdevsec/nginx                          T-Pot Nginx                                     0
vmware/nginx                                                                              3
paketobuildpacks/nginx                                                                    0
gluufederation/nginx                       A customized NGINX image containing a consu…   1
cleanstart/nginx                          Secure by Design, Built for Speed, Hardened …   0
activestate/nginx                         ActiveState's customizable, low-to-no vulner…   0
antrea/nginx                              Nginx server used for Antrea e2e testing        0
intel/nginx                                                                               0
docksal/nginx                             Nginx service image for Docksal                 1
geokrety/nginx                            Our customized nginx image                      0
circleci/nginx                            This image is for internal use                  2
ilios/nginx                               Nginx customized to run Ilios along with the…   0
corpusops/nginx                           https://github.com/corpusops/docker-images/     1
dockette/nginx                            Nginx SSL / HSTS / HTTP2                        3
wayofdev/nginx                            Nginx Docker image for PHP development. SSL-…   0
```

17. docker pull nginx

```
C:\Users\Lab 203>docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
Digest: sha256:5a88c9c45479443d7be2eadc894b4ed0a9801bae03d97a5760ae13b5c2005
942
Status: Image is up to date for nginx:latest
docker.io/library/nginx:latest
```

18. docker run -d --name my_webserver nginx

```
C:\Users\Lab 203>docker run -d --name my_webserver nginx
e1fa24d989ca8eed7ac56a01356a851b1c894e1e9141a5fc8483d3049c778f28
```

19. docker exec -it my_webserver sh

```
C:\Users\Lab 203>docker exec -it my_webserver sh
# exit
```

20. docker stop my_webserver
    docker rm my_webserver

```
C:\Users\Lab 203>docker stop my_webserver
my_webserver
```

21. docker rmi nginx
    docker images

```
C:\Users\Lab 203>docker rmi nginx
Untagged: nginx:latest
Deleted: sha256:5a88c9c45479443d7be2eadc894b4ed0a9801bae03d97a5760ae13b5c200
5942

C:\Users\Lab 203>docker images
                                                    i Info →  U  In Use
IMAGE                  ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-docker:latest    50ffa1e3a0bd       1.6GB         411MB    U
hello-world:latest     c3cbe1cc1aa5      25.9kB        9.49kB    U
new_img_name:latest    d0f28c79138d       202MB        53.2MB
sample69:latest        9fc8bb8bc4fe       1.57GB        395MB    U
ubuntu:20.04           8feb4d8ca535       111MB        29.3MB    U
ubuntu:latest          3131b4cc82a7       161MB        45.3MB    U
```

22. docker pull mysql

```
C:\Users\Lab 203>docker pull mysql
Using default tag: latest
latest: Pulling from library/mysql
81b3a6a3bce8: Pull complete
1576763b528a: Pull complete
61d49498d3da: Pull complete
c2fb254fc2f4: Pull complete
5ac73e0e3f6a: Pull complete
88745f4755f0: Pull complete
e0607e526570: Pull complete
e21df1635f12: Pull complete
bb49c3208fe7: Pull complete
08b808cb6d40: Pull complete
1f9f19898688: Download complete
a701ef10ab46: Download complete
Digest: sha256:8fdf311514c91fa5014e93e98e19d7f5d9eb162a462c13100c956dacc278e
e21
Status: Downloaded newer image for mysql:latest
docker.io/library/mysql:latest
```

Conclusion:

In this experiment, the foundational concepts of Docker and application containerization were successfully studied and practically applied. The theoretical framework established the role of Docker in providing standardized, isolated environments, while clearly distinguishing between static, immutable Docker images and active, writable Docker containers.

Through the hands-on implementation phase, a complete container lifecycle was executed using a standard database image (redis,nginx). The practical steps successfully demonstrated how to:

Search for and pull specific images from a registry.

Deploy an image into an active, isolated container running in the background.

Monitor, inspect, and access the internal shell of a live container.

Perform proper environment cleanup by stopping the container, removing it, and deleting the underlying image.

Ultimately, this experiment provided the working knowledge necessary to package, deploy, and manage isolated software environments consistently across different systems.